



Name: _____

Score: _____ / 100

Date: _____

Course/Year/Section: _____

Midterm Exam - END-TO-END SYSTEM ENGINEERING
CPE 304L – Software Design (Laboratory)

SECTION I: GENERAL PROCEDURES & EXAM INSTRUCTIONS

1. Group Collaboration & Assignments: This examination is a collaborative group assessment. Students must work in their originally assigned mini-system laboratory groups. Each group is strictly required to analyze and design the specific mini-system assigned to them. Changing or swapping mini-systems is strictly prohibited.

2. Core Engineering Bounds: All designs and documentation must strictly reflect the Minimum Viable Product (MVP) standard. Requirements and designs must specify **WHAT** the system does, never **HOW** it is implemented technically. Do not include specific programming languages, non-standard frameworks, or concrete hosting services. Maintain absolute design independence.

3. Submission Deliverables: Each group must submit a unified technical compilation document containing three distinct phases:

- **Phase 1: Requirements Engineering (Mini-SRS):** Consisting of the Problem Statement (standardized syntax), 5 Whys Root Cause Analysis, Scope (Inclusions/Exclusions), 2 distinct User Classes, 3 to 5 Functional Requirements (FRs using 'The system shall...'), and 2 to 3 mathematically testable Non-Functional Requirements (NFRs).
- **Phase 2: Structural Modeling (UML Class Diagram):** A complete UML Class Diagram containing at least 4 key classes with explicitly defined attributes (with physical data types), operations/methods (with return types), associations (Aggregation, Composition, Generalization, or Association), multiplicities, and a 2-3 sentence logic justification.
- **Phase 3: Database Engineering & Data Schema:** Consisting of a Data Dictionary Matrix, a Logical Entity-Relationship Diagram (ERD) using Crow's Foot notation mapping multiplicities to cardinalities, and a Physical Schema Translation Table with physical SQL constraints (NOT NULL, UNIQUE, Primary Keys, Foreign Keys, and referential actions).

SECTION II: THE ASSIGNABLE MINI-SYSTEMS & SYMPTOMS

Every group must locate their originally picked mini-system below. The listed stakeholder symptom and scenario MUST serve as your engineering baseline.

1. Task Reminder System

- **Stakeholder Symptom:** Symptom: Users report missing high-priority deadlines despite receiving regular system notifications.
- **System Scenario:** Scenario: The system broadcasts automated notifications for all tasks at identical decibel and vibration volumes. This results in heavy user 'notification fatigue' where users dismiss or ignore all incoming reminders, failing to recognize critical tasks.

2. Attendance Monitoring System

- **Stakeholder Symptom:** Symptom: Course instructors report that manual roll calls waste 15% of total class lecture time.
- **System Scenario:** Scenario: In a 60-minute classroom session, passing around a paper-based attendance sheet and manually transcribing names consumes 10 full minutes. Sheets are regularly mislaid, and student handwriting is frequently unreadable.

3. Student Consultation Scheduler

- **Stakeholder Symptom:** Symptom: Students regularly travel to campus only to find that their instructor is unavailable.
- **System Scenario:** Scenario: Students schedule consultation times on a static physical board, but professors frequently get pulled into unexpected department meetings. Because the calendar is physical and static, there is no real-time status synchronization.

4. Visitor Log Management System

- **Stakeholder Symptom:** Symptom: Facility security guards cannot verify the real-time headcount of external visitors inside the building.
- **System Scenario:** Scenario: Visitors sign a paper logbook at the entry gate. When visitors exit, they frequently bypass signing out. Security guards have no structural mechanism to reconcile entries and exits, presenting a severe security hazard during emergencies.

5. Meeting Room Reservation System

- **Stakeholder Symptom:** Symptom: Student organizations constantly experience scheduling conflicts and arguments over room usage.
- **System Scenario:** Scenario: There is a physical calendar taped to the room door. Student leaders write reservations in pencil; rival groups regularly erase, alter, or write over existing names, leading to scheduling collisions.

6. Personal Budget Manager

- **Stakeholder Symptom:** Symptom: Users report feeling consistently cash-strapped at month-end without understanding where their funds went.
- **System Scenario:** Scenario: The system records total monthly expenditures as raw numeric data. However, because it completely lacks spending categorization, users are unable to run analysis or change non-essential spending habits.

7. Inventory Monitoring System

- **Stakeholder Symptom:** Symptom: Core administrative office supplies (such as printer toner) deplete fully before replacements are ordered.
- **System Scenario:** Scenario: Office personnel check physical inventory stocks only on Friday afternoons. If a large document project consumes all available toner cartridges on a Monday morning, operations stall for five working days.

8. Student Portfolio Management System

- **Stakeholder Symptom:** Symptom: Senior-year students lose critical academic projects permanently due to local drive crashes or theft.
- **System Scenario:** Scenario: There is no central, verified repository for student projects. Students save files locally on USB flash drives or laptops, leading to massive data losses that compromise their graduation portfolios.

9. Lost & Found Management System

- **Stakeholder Symptom:** Symptom: 90% of retrieved items are never claimed by owners and are eventually discarded.
- **System Scenario:** Scenario: Found items are locked inside a physical box at the security room. Students rarely travel to the administrative building to check the box manually, resulting in massive piles of unclaimed electronics and keys.

10. Concert Event Ticketing System

- **Stakeholder Symptom:** Symptom: Widespread ticket scalping, bot manipulation, and identity fraud bypass gate security.
- **System Scenario:** Scenario: Organizers lessened physical lines via online queuing, but scalpers deploy automated bot networks to purchase VIP tickets and resell them at 300% markups. Furthermore, because venues experience severe wireless network congestion, gate security relies on physical/static paper tickets. Since ticket codes are not bound to physical identities, compromised or duplicate tickets cannot be revoked offline, allowing unauthorized entries.

SECTION III: TECHNICAL REFERENCE GUIDE & STUDENT PATTERN

To ensure strict technical compliance, use the following completed solution for the **Meeting Room Reservation System** as your structural reference.

PHASE 1: REQUIREMENTS ENGINEERING (MINI-SRS)

1. Problem Discovery (5 Whys Analysis):

- Symptom: Student organizations argue over double-booked rooms.
- Why 1: Why do they argue? Because multiple groups arrive at the same room expecting exclusive access.
- Why 2: Why do multiple groups expect access? Because their names are written on the same calendar slot.
- Why 3: Why are multiple names written on the same slot? Because previous names were erased and overwritten.
- Why 4: Why were they erased and overwritten? Because the booking calendar is a physical sheet of paper taped to the door, accessible to anyone with an eraser or pencil.
- Root Cause: The system lacks central, tamper-proof reservation storage and validation to prevent unauthorized write access.

2. Standardized Problem Statement:

Student Organization Leaders need a tamper-proof, centralized reservation coordination system because the physical door calendar is vulnerable to physical alteration, causing scheduling collisions and organizational conflicts.

3. Scope Boundaries:

- **Inclusions (In-Scope):** Real-time read-only schedule visualization, digital reservation requests, and programmatic collision detection (preventing overlapping booking entries).
- **Exclusions (Out-of-Scope):** Physical door lock automation, general academic class timetabling, and catering or equipment booking services.

4. User Classes:

- **Student Org Leader:** Authenticates, views real-time room availability, and submits digital booking requests.
- **Room Administrator:** Reviews pending booking requests, issues approvals or rejections, and manages the room inventory list.

5. Minimum Viable Product (MVP) Requirements:

- **Functional Requirements (FRs):**
 - FR-001: The system shall display a read-only calendar interface of all approved room bookings to unauthenticated and authenticated users.
 - FR-002: The system shall automatically reject any reservation request that overlaps in date and time with an existing approved reservation slot.
 - FR-003: The system shall require the Room Administrator's digital authorization before updating a reservation status from 'Pending' to 'Approved'.

- **Non-Functional Requirements (NFRs):**

- NFR-001 (Security): The system shall authenticate Student Org Leaders using the university's standardized LDAP credentials prior to allowing reservation submission.
- NFR-002 (Availability): The digital reservation calendar must be accessible via standard web browsers 24/7 with a minimum measured uptime of 99.9% per academic term.

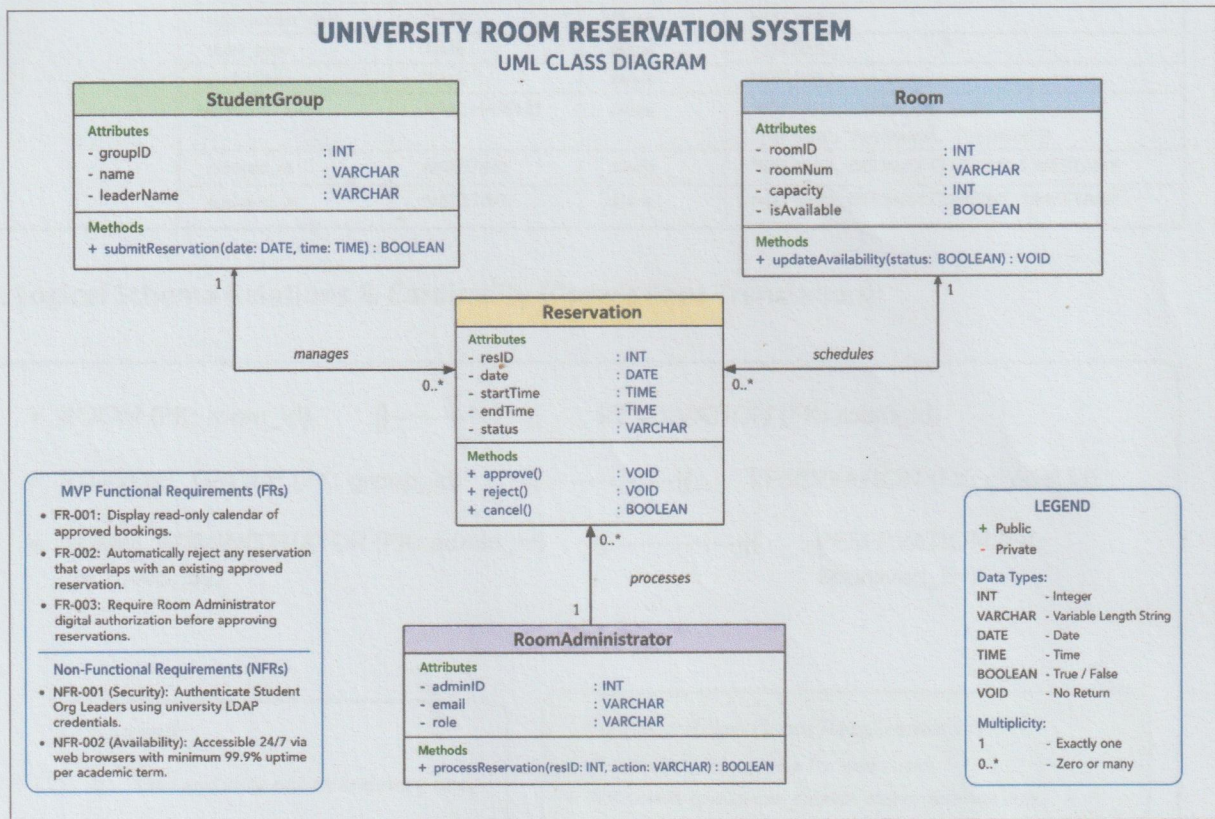
PHASE 2: STRUCTURAL MODELING (UML CLASS DIAGRAM)

1. **Class Specifications:** Define at least 4 key classes. Attributes and methods must include explicit data types and return types.

- **StudentGroup Class:**
- **Room Class:**
- **Reservation Class:**
- **RoomAdministrator Class:**

2. **Structural Relationships & Multiplicity:**

- StudentGroup [1] ----- manages -----> [0..*] Reservation
- Reservation [0..*] <----- schedules ----- [1] Room
- RoomAdministrator [1] ----- processes -----> [0..*] Reservation



3. Logic Justification (How it prevents the symptom):

"The *Reservation* class is structurally decoupled from *StudentGroup* and bounded strictly to a unique *Room*. This 1-to-1 relationship between an approved reservation slot and a room, combined with database-level uniqueness validation, prevents scheduling collisions and makes reservations tamper-proof."

PHASE 3: DATABASE ENGINEERING & DATA MODELING

1. Data Dictionary Matrix:

Table Name	Column Name	Data Type	Key Type	Validation / Constraints
room	room_id	INT	PK	AUTO_INCREMENT
	room_number	VARCHAR(10)	None	NOT NULL, UNIQUE
	capacity	INT	None	NOT NULL, > 0
	is_available	BOOLEAN	None	NOT NULL, DEFAULT TRUE
student_group	group_id	INT	PK	AUTO_INCREMENT
	name	VARCHAR(100)	None	NOT NULL
	leader_name	VARCHAR(100)	None	NOT NULL
room_administrator	admin_id	INT	PK	AUTO_INCREMENT
	email	VARCHAR(100)	None	NOT NULL, UNIQUE
	role	VARCHAR(50)	None	NOT NULL
reservation	reservation_id	INT	PK	AUTO_INCREMENT
	room_id	INT	FK	NOT NULL, REFERENCES room(room_id)
	group_id	INT	FK	NOT NULL, REFERENCES student_group(group_id)
	approved_by	INT	FK	REFERENCES room_administrator(admin_id), NULL
	reservation_date	DATE	None	NOT NULL
	start_time	TIME	None	NOT NULL
	end_time	TIME	None	NOT NULL
	status	VARCHAR(15)	None	NOT NULL, CHECK (status IN ('Pending', 'Approved', 'Cancelled'))
	created_at	DATETIME	None	NOT NULL, DEFAULT CURRENT_TIMESTAMP
	updated_at	DATETIME	None	NOT NULL, DEFAULT CURRENT_TIMESTAMP

2. Logical Schema Relations & Cardinality (Crow's Foot Translation):

- ROOM (PK: room_id) ||-----|{ RESERVATION (FK: room_id)
- STUDENT_GROUP (PK: group_id) ||-----|{ RESERVATION (FK: group_id)
- ROOM_ADMINISTRATOR (PK: admin_id) ||-----|{ RESERVATION (FK: approved_by)

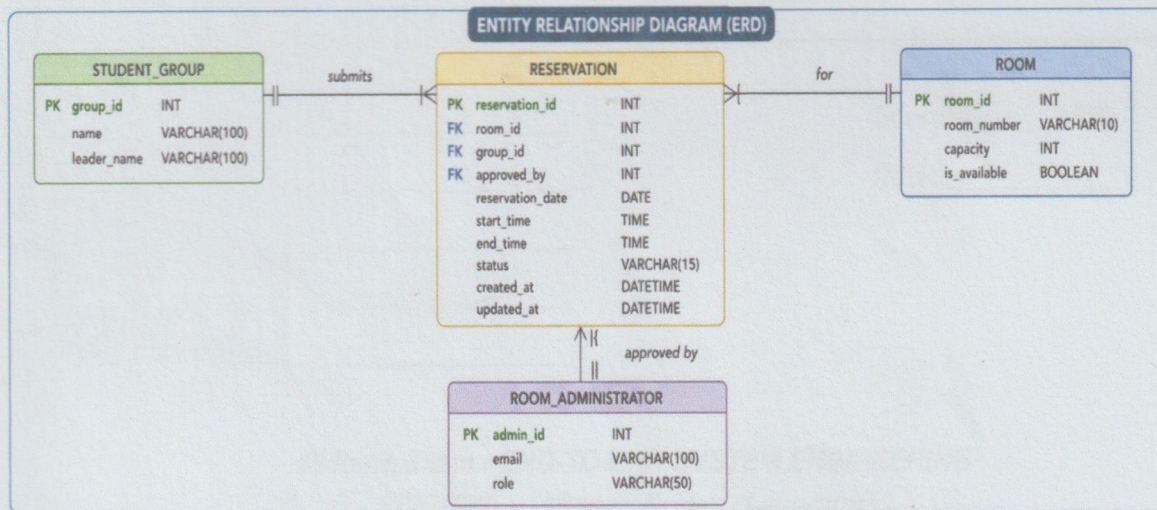
Legend:

- || One and only one (mandatory one)
- |{ One or many (zero or many)
- PK Primary Key
- FK Foreign Key

Business Rules (from Requirements)

- A reservation must be for one room.
- A student group can submit many reservations.
- A reservation must be approved by one administrator (approved_by can be NULL while pending).
- Overlapping reservations for the same room are prevented by business logic (FR-002).

3. Entity Relation Diagram (ERD)



SECTION IV: COMPREHENSIVE EVALUATION RUBRIC

The examination will be graded using the official criteria below. Technical precision, logical consistency between phases, and adherence to structural conventions are prioritized.

Evaluation Phase	Weight	Exemplary Standards (Score Range: 90% - 100%)
Phase 1: Requirements Engineering (Mini-SRS)	35%	The 5 Whys process is perfectly logical, leading to a structural root cause. The problem statement strictly follows the mandated [User] needs [Goal] because [Cause] syntax. The FRs (exactly 3 to 5) are testable and use the 'The system shall...' syntax. The NFRs (exactly 2 to 3) specify measurable, quantitative metrics.
Phase 2: Structural Modeling (UML Class Diagram)	30%	Includes at least 4 key classes with complete, syntactically correct attributes (with data types) and methods (with return types). Relationships (Association, Aggregation, Composition, Inheritance) are logically mapped with correct multiplicity notations. The logic justification is 2-3 sentences and structurally matches the requirements.
Phase 3: Database Engineering & Data Schema	25%	Tables are correctly converted from classes, and methods are stripped. Primary and Foreign Keys are correctly designated and linked to establish clean relationships. Cardinalities in Crow's Foot are perfectly precise (e.g., 1:N, 1:1). Table constraints (NOT NULL, UNIQUE, CHECK, FK) are explicitly defined and prevent redundancy.
Technical & Document Professionalism	10%	Consistent naming conventions (PascalCase for UML, snake_case for Database). Structural consistency is maintained across all three phases (e.g., the same system entities exist in SRS, UML, and SQL Schema). Zero grammatical/spelling errors, and zero usage of forbidden metaphors.
TOTAL SCORE	100%	Excellent engineering work showing logical consistency, rigor, and system modeling skills.